

PL/SQL

Asst. Prof. Esmita Gupta

SQL vs. PL/SQL

- SQL is the natural language of the DBA, but it suffers from various inherent disadvantages, when used as a conventional programming language.
- SQL does not have any procedural capabilities i.e. SQL does not provide the programming techniques or condition checking, looping and branching that is vital for data testing before its permanent storage.
- SQL statement is executed when a call is made to the engines resource. This adds to the traffic on the network, thereby decreasing the speed of data processing, especially in multi-user environment.
- While processing an SQL sentence if an error occurs, the Oracle engine displays its own error messages.
- SQL has no facility for programmed handling of errors that arise during the manipulation of data.

What is PL/SQL

- **PL/SQL** stands for Procedural Language extension of SQL.
- PL/SQL is a combination of SQL along with the procedural features of programming languages.
- It was developed by Oracle Corporation in the early 90's to enhance the capabilities of SQL.

The PL/SQL Engine:

- Oracle uses a PL/SQL engine to processes the PL/SQL statements. A PL/SQL code can be stored in the client system (client-side) or in the database (server-side).

Advantages of PL/SQL

- PL/SQL is development tool that only supports SQL data manipulation but also provides facilities of conditional checking, branching and looping.
- PL/SQL sends an entire block of SQL statements to the Oracle engine all in one go.
- Communication between the program block and the Oracle engine reduces considerably, reducing network traffic. Since the Oracle engine got the SQL statements as a single block, it processes this code much faster than if it got the code one sentence of a time.
- There is a definite improvement in the performance time of the Oracle engine. As an entire block of SQL code is passed to the Oracle engine at one time for execution, all changes made to the data in the table are done or undone, in one go.

Cont...

- PL/SQL also permits dealing with errors as required, and facilitates displaying user-friendly messages, when errors are encountered.
- PL/SQL allows declaration and use of variables in blocks of code. These variables can be used to store intermediate result of a query for later processing or calculate values and insert them in to an Oracle table later.
- PL/SQL variable can be used anywhere either in SQL statements or in PL/SQL block via PL/SQL block engine. This considerably improves transaction performance applications system, where Oracle is operational.
- Hence, PL/SQL code blocks written for a DOS version of Oracle will run on its Linux/Unix version, without any modifications at all.

PL/SQL Block

- **PL/SQL Block consists of three sections:**
 - The Declaration section (optional).
 - The Execution section (mandatory).
 - The Exception (or Error) Handling section (optional).

Declaration Section:

The Declaration section of a PL/SQL Block starts with the reserved keyword DECLARE. This section is optional and is used to declare any placeholders like variables, constants, records and cursors, which are used to manipulate data in the execution section. Placeholders may be any of Variables, Constants and Records, which stores data temporarily. Cursors are also declared in this section.

Cont...

Execution Section:

The Execution section of a PL/SQL Block starts with the reserved keyword BEGIN and ends with END. This is a mandatory section and is the section where the program logic is written to perform any task. The programmatic constructs like loops, conditional statement and SQL statements form the part of execution section.

Cont...

- **Exception Section:**

The Exception section of a PL/SQL Block starts with the reserved keyword `EXCEPTION`. This section is optional. Any errors in the program can be handled in this section, so that the PL/SQL Blocks terminates gracefully. If the PL/SQL Block contains exceptions that cannot be handled, the Block terminates abruptly with errors.

PL/SQL Block Looks

- Every statement in the above three sections must end with a semicolon ; . PL/SQL blocks can be nested within other PL/SQL blocks. Comments can be used to document code.

How a Sample PL/SQL Block Looks

```
DECLARE
    Variable declaration
BEGIN
    Program Execution
EXCEPTION
    Exception handling
END;
```

Example:

```
DECLARE
    var_salary number(6);
    var_emp_id number(6) := 1116;
BEGIN
    SELECT salary
    INTO var_salary
    FROM employee
    WHERE emp_id := var_emp_id;
    dbms_output.put_line(var_salary);
    dbms_output.put_line('The employee '
    || var_emp_id || ' has salary ' ||
    var_salary);

END;
```

PL/SQL Program Commands

```
--PL/SQL program to display the current date
DECLARE
    todays_date DATE;
BEGIN
    todays_date := SYSDATE;
    DBMS_OUTPUT.PUT_LINE('Today''s date is ');
    DBMS_OUTPUT.PUT_LINE(todays_date);
END;
```

Figure 4-3 PL/SQL program commands

Output: **Today's data is**
 18-OCT-07

PL/SQL procedure successfully completed

- If no output appears use: **SQL> set serveroutput on**

PL/SQL Placeholders

- Placeholders are temporary storage area. Placeholders can be any of Variables, Constants and Records. Oracle defines placeholders to store data temporarily, which are used to manipulate data during the execution of a PL SQL block.
- Depending on the kind of data you want to store, you can define placeholders with a name and a datatype. Few of the datatypes used to define placeholders are as given below.
Number (n,m) , Char (n) , Varchar2 (n) , Date , Long , Long raw, Raw, Blob, Clob, Nclob, Bfile

Composite Variables

- **Data Structure** is a data object made up of multiple individual data elements
- **Composite variable** is a data structure contains multiple scalar variables
- Types:
 - RECORD
 - TABLE
 - VARRAY

Reference Variables

- Reference variables directly reference specific database column or row
- Reference variables assume data type of associated column or row
- %TYPE data declaration syntax:

*variable_name
tablename.fieldname%TYPE;*

- %ROWTYPE data declaration syntax:

variable_name tablename%ROWTYPE;

Reference Variables

- The (%TYPE) reference data type specifies a variable that references a single DB field.

`last_name FACULTY.F_LAST%TYPE;`

- The last_name variable assumes a data type of VARCHAR2(30), because this is the data type of the f_last column in the FACULTY table.

Reference Variables

- The (%ROWTYPE) reference data type creates composite variables that reference the entire data record.

```
Faculty_row FACULTY%ROWTYPE;
```

- The variable `faculty_row` references all of the columns in the `FACULTY` table, and each column has the same data type as its associated DB column.

PL/SQL Variables

General Syntax to declare a var is

var_name datatype [NOT NULL :=
value];

- *variable_name* is the name of the variable.
- *datatype* is a valid PL/SQL datatype.
- NOT NULL is an optional specification on the variable.
- *value* or DEFAULT *value* is also an optional specification, where you can initialize a variable.
- Each variable declaration is a separate statement and must be terminated by a semicolon.

```
DECLARE
salary number(4);
dept varchar2(10) NOT NULL :=
    "HR Dept";

BEGIN
Salary :=200;
END;

/
```

PL/SQL Constants

- As the name implies a constant is a value used in a PL/SQL block that remains unchanged throughout the program.
- A constant is a user-defined literal value.
- We can declare a constant and use it instead of actual value.
- For example if we want to write a program which will increase the salary of the employees by 25%, we can declare a constant and use it throughout the program. Next time when we want to increase the salary again we can change the value of the constant which will be easier than changing the actual value throughout the program.

For example

- The general syntax to declare a constant is:
 `constant_name CONSTANT datatype := VALUE;`
- `constant_name` is the name of the constant i.e. similar to a variable name.
- The word `CONSTANT` is a reserved word and ensures that the value does not change.
- `VALUE` - It is a value which must be assigned to a constant when it is declared. We cannot assign a value later.
- For example, to declare `salary_increase`, we can write code as follows:
 `DECLARE`
 `salary_increase CONSTANT number (3) := 10;`

Assignment Statements

- Assigns value to variable
- Operator

:=

- Syntax

variable_name := value;

- String literal must be enclosed in a single quotation marks.

Displaying PL/SQL Program Output in SQL*Plus

- Display program output

```
DBMS_OUTPUT.PUT_LINE('display_text');
```

- Display maximum of 255 characters of text data.
- More than 255 characters gives an error.

- Example:

```
s_name := 'Ahmed Mahmoud';
```

```
DBMS_OUTPUT.PUT_LINE(s_name);
```

PL/SQL EXPRESSIONS AND COMPARISONS

- Expressions are constructed using operands and operators. An operand is a variable, constant, literal, or function call that contributes a value to an expression
- **Logical operators**
The logical operators AND, OR, and NOT follow the tri-state logic(True, False, Null)
- **Boolean operations:** In a SQL statement, BOOLEAN expressions let us specify the rows in a table that are affected by some statements. In a procedural statement, BOOLEAN expressions are the basis for conditional control.
- There are three kinds of BOOLEAN expressions: arithmetic, character, and date.

- **BOOLEAN Arithmetic Expressions:** We can use the relational operators to compare numbers for equality or inequality.
- For eg. number1 := 75;
 number2 := 70;

The following expression is true:

number1 > number2

- **BOOLEAN Character Expressions:** We can compare character values for equality or inequality. By default, comparisons are based on the binary values of each byte in the string.
- For example, given the assignments
 string1 := 'Kathy';
 string2 := 'Kathleen';
 The following expression is true:
 string1 > string2

Cont...

- **BOOLEAN Date Expressions:** We can also compare dates. Comparisons are chronological; that is, one date is greater than another if it is more recent.
- For example,
 - given the assignments
 - date1 := '01-JAN-91';
 - date2 := '31-DEC-90';
 - The following expression is true:
 - date1 > date2

CASE Expression

CASE Expression: There are two types of expressions used in CASE statements: simple and searched. These expressions correspond to the type of CASE statement in which they are used.

- **Simple CASE expression:** A simple CASE expression selects a result from one or more alternatives, and returns the result. Although it contains a block that might stretch over several lines, it really is an expression that forms part of a larger statement, such as an assignment or a procedure call. The CASE expression uses a selector, an expression whose value determines which alternative to return.

Example:

```
DECLARE
deptno NUMBER := 20;
dept_desc VARCHAR2(20);
BEGIN
dept_desc := CASE deptno
WHEN 10 THEN 'Accounting'
WHEN 20 THEN 'Research'
WHEN 30 THEN 'Sales'
WHEN 40 THEN 'Operations'
ELSE 'Unknown'
END;
DBMS_OUTPUT.PUT_LINE(dept_desc);
END;
```

- **Searched CASE Expression:** A searched CASE expression lets us test different conditions instead of comparing a single expression to various values. A searched CASE expression has no selector.
- Each WHEN clause contains a search condition that yields a BOOLEAN value, so we can test different variables or multiple conditions in a single WHEN clause.

```
DECLARE
```

```
sal NUMBER := 2000;
```

```
sal_desc VARCHAR2(20);
```

```
BEGIN
```

```
sal_desc := CASE
```

```
    WHEN sal < 1000 THEN 'Low'
```

```
    WHEN sal BETWEEN 1000 AND 3000 THEN 'Medium'
```

```
    WHEN sal > 3000 THEN 'High'
```

```
    ELSE 'N/A'
```

```
END;
```

```
DBMS_OUTPUT.PUT_LINE (sal_desc);
```

```
END;
```

NULL Value in Comparisons

- When working with nulls, we can avoid some common mistakes by keeping in mind the following rules:
- Comparisons involving nulls always yield NULL
- Applying the logical operator NOT to a null yields NULL
- In conditional control statements, if the condition yields NULL, its associated sequence of statements is not executed.
- If the expression in a simple CASE statement or CASE expression yields NULL, it cannot be matched by using WHEN NULL. In this case, we would need to use the searched case syntax and test WHEN expression IS NULL.
- Example:

```
IF rating > 90 THEN
  compute_bonus(emp_id);
ELSE
  NULL;
END IF;
```

PL/SQL Data types

- **Number(precision,scale)**

precision, is the total number of digits, and scale, which is the number of digits to the right of the decimal point.

- **CHAR (maximum_size)**

The CHAR datatype takes an optional parameter that lets us specify a maximum size up to 32767 bytes. We can specify the size in terms of bytes or characters.

- **VARCHAR2(size)**

you specify a maximum string length (in bytes or characters) between 1 and 4000 bytes for the VARCHAR2 column.

- **Boolean type:**

BOOLEAN datatype is used to store the logical values TRUE, FALSE, and NULL (which stand for a missing, unknown, or inapplicable value)

- **Datetime type:** The Datetime datatypes lets us store and manipulate dates, times, and intervals (periods of time). The date function SYSDATE returns the current date and time.

CONTROL STRUCTURES

- Control structures are meant to control the flow of program in a programming language.
- There are three types of control structures:
 - Conditional control
 - Iterative control
 - Sequential control

Conditional statement

IF THEN ELSE STATEMENT

- 1) **IF** condition
 THEN statement 1;
 ELSE statement 2;
 END IF;
- 2) IF condition 1
 THEN statement 1;
 statement 2;
 ELSIF condition2
 THEN statement 3;
 ELSE statement 4;
 END IF

- 3) IF condition1 THEN ELSE IF
 condition2
 THEN statement1;
 END IF;
 ELSIF condition3
 THEN statement2;
 END IF;

For example

Declare

```
y number(5,2);
```

Begin

```
y := &y;
```

```
if y >= 70 then
```

```
dbms_output.put_line('Distinction');
```

```
elsif y >= 60 then
```

```
dbms_output.put_line('First class');
```

```
elsif y >= 50 then
```

```
dbms_output.put_line('Second class');
```

```
elsif y >= 35 then
```

```
dbms_output.put_line('Passed');
```

```
else
```

```
dbms_output.put_line('Failed');
```

```
end if;
```

```
end;
```

- 1) WAP to initialize two numbers and display them.
- 2) WAP to check whether the number is greater than 20 or less than 20 and display the data.

CASE statement

- The PL/SQL CASE statement allows you to execute a sequence of statements based on a selector variable.

The syntax for CASE statement is

```
CASE < selector variable >
```

```
WHEN < expression 1 > THEN
```

```
< commands >
```

```
WHEN < expression 2 > THEN
```

```
< commands >
```

```
.
```

```
.
```

```
.
```

```
[ELSE
```

```
< commands >]
```

```
END CASE
```

Example

```
DECLARE grade char(1) := 'A';  
BEGIN  
CASE grade  
when 'A' then dbms_output.put_line('Excellent');  
when 'B' then dbms_output.put_line('Very good');  
when 'C' then dbms_output.put_line('Well done');  
when 'D' then dbms_output.put_line('You passed');  
when 'F' then dbms_output.put_line('Better try again');  
else  
dbms_output.put_line('No such grade');  
END CASE;  
END;  
/
```

ITERATIVE CONTROL

- Iterative control statements are meant for a sequence of statements that has to be repeated. In PL/SQL there are three ways for iterative statements :

1. **LOOP and EXIT**

2. **WHILE LOOP**

3. **FOR LOOP**

LOOP...EXIT

- The EXIT statement forces a loop to complete unconditionally. When an EXIT statement is encountered, the loop completes immediately and control passes to the next statement:

- Syntax:

```
LOOP  
  < statements >  
  [EXIT WHEN < condition >;]  
END LOOP;
```

- *LOOP*

IF counter < 3 THEN

EXIT;

END IF;

counter := counter - 1;

END LOOP;

- In this case, the program exits loop immediately when counter is less than 3. If the counter starts with a number less than 3, all statements after that won't execute even once. Remember, the EXIT statement must be placed inside a loop.

LOOP...EXIT WHEN

- Unlike the LOOP...EXIT statement, the LOOP...EXIT-WHEN statement lets a loop complete conditionally. When the EXIT statement is encountered, the condition in the WHEN clause is evaluated. If the condition is true, the loop completes and control passes to the next statement after the loop. If the condition is false, the loop continues.
- Eg. LOOP
 monthly_value := daily_value * 31;
 EXIT WHEN monthly_value > 4000;
END LOOP;

WHILE LOOP

- A **WHILE LOOP** statement in PL/SQL programming language repeatedly executes a target statement as long as a given condition is true.
- The syntax for WHILE LOOP is as follows:

```
WHILE < condition >  
LOOP  
  < commands >  
END LOOP;
```

For example:

```
DECLARE  
a number(2) := 10;  
BEGIN  
  WHILE a < 20 LOOP  
    dbms_output.put_line('value  
                           of a: ' || a);  
    a := a + 1;  
  END LOOP;  
END;  
/
```

FOR LOOP

- A **FOR LOOP** is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

- The syntax for FOR LOOP is:

```
FOR variable_name IN  
    [REVERSE] start..end  
LOOP  
< commands >  
END LOOP;
```

- For example:

```
FOR i IN 1..3  
LOOP  
    DBMS_OUTPUT.PUT_LINE(i);  
END LOOP;
```

```
FOR i IN 3..1  
LOOP  
    DBMS_OUTPUT.PUT_LINE(i);  
END LOOP;
```

SEQUENTIAL CONTROL

- **GOTO Statement**

The GOTO statement transfers control to a label unconditionally. The label must be unique in its scope and must precede an executable statement or a PL/SQL block. When run, the GOTO statement transfers control to the labelled statement or block.

```
DECLARE
a number(2) := 10;
BEGIN
<<loopstart>>
  -- while loop execution
  WHILE a < 20
  LOOP
    dbms_output.put_line ('value of a: ' || a);
    a := a + 1;
    IF a = 15 THEN
      a := a + 1;
      GOTO loopstart;
    END IF;
  END LOOP;
END;
/
```

Restrictions with GOTO Statement

GOTO Statement in PL/SQL imposes the following restrictions:

- A GOTO statement cannot branch into an IF statement, CASE statement, LOOP statement or sub-block.
- A GOTO statement cannot branch from one IF statement clause to another or from one CASE statement WHEN clause to another.
- A GOTO statement cannot branch from an outer block into a sub-block (that is, an inner BEGIN-END block).
- A GOTO statement cannot branch out of a subprogram. To end a subprogram early, either use the RETURN statement or have GOTO branch to a place right before the end of the subprogram.
- A GOTO statement cannot branch from an exception handler back into the current BEGIN-END block. However, a GOTO statement can branch from an exception handler into an enclosing block.

Continue statement

The **CONTINUE** statement causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating. In other words, it forces the next iteration of the loop to take place, skipping any code in between.

Syntax:

Continue;

```
DECLARE
a number(2) := 10;
BEGIN -- while loop execution
WHILE a < 20 LOOP
  dbms_output.put_line ('value of a: '
                        || a);

  a := a + 1;
  IF a = 15 THEN
    a := a + 1;
    CONTINUE;
  END IF;
END LOOP;
END;
/
```

Queries

- WA PL/SQL code block that will accept an account number from the user, check if the user balance is less than the minimum balance, only then deduct Rs.100/- from the balance. The process is fired on account table.
- WA PL/SQL code for a simple loop such that a message is displayed when a loop exceeds a particular value(Assume loop starts at 0 and ends at 10).
- WA PL/SQL code to calculate the area of a circle.
- WA PL/SQL code to calculate the area of a circle of radius varying from 3 to 7. Store the radius and the corresponding values of calculated area in an empty table named AREA, consisting of two cols Radius and Area.

Cont...

- Assume sales table consisting of column zone, prod_id, quantity. Write a PL/SQL code to accept zone and product_id from user to display total sale of specified product and zone.
- WA PL/SQL code to accept empno and salary of an employee. Check for the empno and find the job using the value of the salary and store it in the table accordingly by using case... when stmt.

The name of the table used is emp_jobs(empno, job).

1. salary >= 8000 and salary <= 10000 then "clerk"
2. Salary > 10000 and salary <= 20000 then "Officer"
3. Salary > 20000 then "Asst. Manager."

Consider the program below, which processes a bank transaction. Before allowing you to withdraw \$500 from account 3, it makes sure the account has sufficient funds to cover the withdrawal. If the funds are available, the program debits the account. Otherwise, the program displays a message

```
DECLARE
  acct_balance NUMBER(11,2);
  acct      CONSTANT NUMBER(4) := 3;
  debit_amt  CONSTANT NUMBER(5,2) := 500.00;
BEGIN
  SELECT bal INTO acct_balance FROM accounts
    WHERE account_id = acct;
  IF acct_balance >= debit_amt THEN
    UPDATE accounts SET bal = bal - debit_amt
      WHERE account_id = acct;
  ELSE
    DOPL('Acct_balance : Insufficient funds');
  END IF;
END;
/
```

To print message whether the given number is odd or even.

```
DECLARE
a NUMBER:=&a;
BEGIN
IF( mod(a,2)=0) THEN
dbms_output.put_line('a is even number' );
ELSE
dbms_output.put_line('a is odd number1);
END IF;
dbms_output.put_line ('Program completed. ');
END;
/
```

To print the grade based on the given marks without else condition (mark ≥ 70 Grade A, mark ≥ 40 and mark < 70 Grade B, mark ≥ 35 and mark < 40 Grade C).

```
DECLARE
mark NUMBER :=55;
BEGIN
dbms_output.put_line('Program started.' );
IF( mark  $\geq 70$ ) THEN
dbms_output.put_line('Grade A');
ELSIF(mark  $\geq 40$  AND mark  $< 70$ ) THEN
dbms_output.put_line('Grade B');
ELSIF(mark  $\geq 35$  AND mark  $< 40$ ) THEN
dbms_output.put_line('Grade C');
END IF;
dbms_output.put_line('Program completed. ');
END;
/
```

To print number from 1 to 5 using basic loop statement.

```
DECLARE
a NUMBER:=1;
BEGIN
dbms_output.put_line('Program started. ');
LOOP
dbms_output.put_line(a);
a:=a+1;
EXIT WHEN a>5;
END LOOP;
dbms_output.put_line('Program completed');
END:
/
```

Write a pl/sql program to count the number of employees in department no.50

```
declare
    tot_emp number;
begin
    select count(*)
    into tot_emp
    from departments
    where department_id = 50;

    dbms_output.put_line('the employees are in the department
                          50: ' || tot_emp);
end;
/
```

Write a pl/sql program to check whether the number is positive, negative or zero

```
declare
num1 number := &get_num;
begin
if num1 < 0 then
dbms_output.put_line ('the number ' || num1 || ' is a negative
number');
elsif num1 = 0 then
dbms_output.put_line ('the number ' || num1 || ' is equal to zero');
else
dbms_output.put_line ('the number ' || num1 || ' is a positive
number');
end if;
end;
/
```

Write a pl/sql program to check whether the number is positive, negative or zero

Declare

v_name char2(20);

v_hiredate date;

v_sal number(7);

Emp_No emp.empno%TYPE;

BEGIN

select ename, hiredate, sal into v_name, v_hiredate, v_sal
from emp

where empno=&Emp_No;

dbms_output.put_line(v_name || ' ' || v_sal || ' ' || v_hiredate);

END;